MODULE 10 · CLASS SLIDES

LLMs, RAGs and AI Foundations

Varun Kumar · SkillNavora

Prepared for Skillnavora · Learn. Practise. Implement. Grow.

Product Management Roadmap

PHASE 1
CRAFT
Modules 1–4

1. Fundamentals of Product Management
2. Product Strategy & Strategic Thinking
3. Problem Discovery, Market & User Research
4. Problem Framing & Prioritization

PHASE 2
CRAFT + AI
Modules 5–8

5. UX & Design Collaboration
6. Effective Communication as a PM
7. Analytics & Metrics
8. Product Launch, Growth & Monetization

PHASE 3
DEEP AI
Modules 9–13

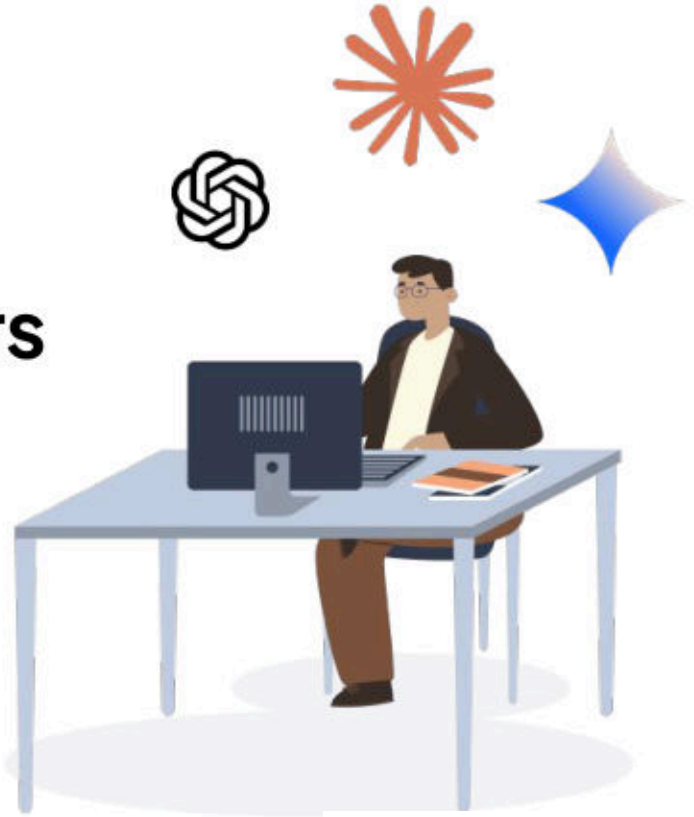
9. Tech 101 & System Design
10. LLMs, RAGs & AI Foundations — WEEK 10
11. Vibe Coding for PMs
12. Agentic AI & AI Systems in Production
13. Capstone Project & Interview Prep

What did we cover last week ?

1. Why technical knowledge matters for PMs
2. Frontend vs. Backend fundamentals
3. Working with APIs and cloud basics
4. Asynchronous systems: Polling vs. Webhooks
5. System design essentials
6. Collaborating effectively with engineering teams
7. Using AI to accelerate technical learning

LLMs for Product Managers

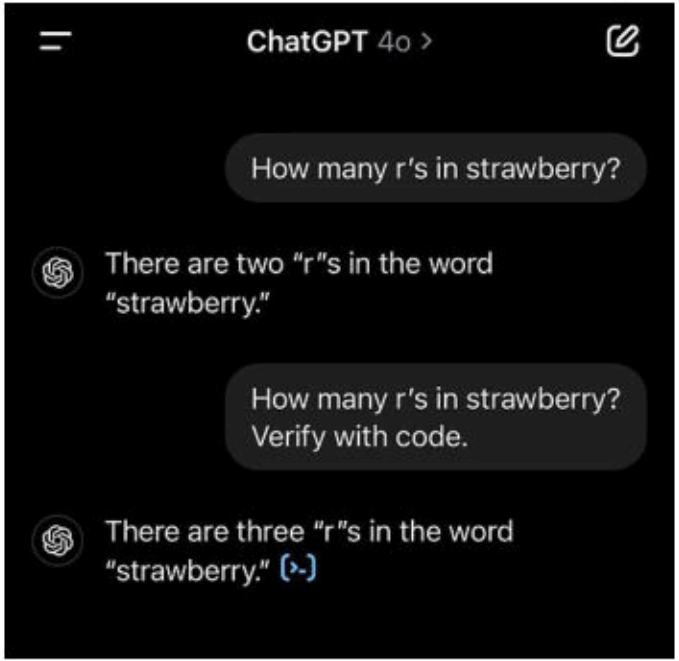
“From black box mystery → product layer you can design for.”



Session 1 Takeaways

- Explain LLM functioning in simple terms
- Identify AI product layers and components
- Use Playground to observe model behavior

LLMs & RAGs for PMs



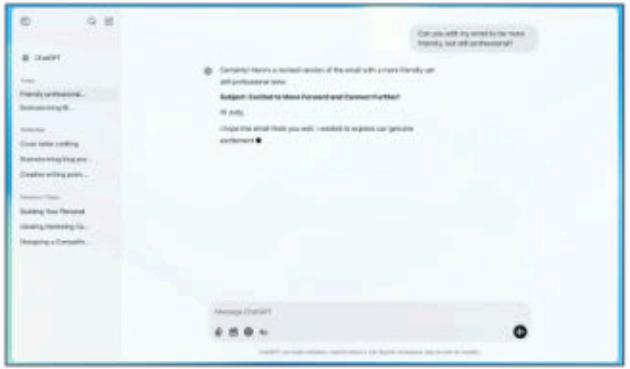
What is fundamentally different?



You search



It recommends



It collaborates

For the first time software can understand natural language.

That's why PMs need to understand LLMs.

Which task is harder?

A) Write a poem about heartbreak.

B) Count the number of chairs in this room.

For AI:

- Writing a poem → Easy
- Counting chairs from a camera feed → Surprisingly difficult

For humans:

- Writing a poem → Hard
- Counting chairs → Trivial

Day 1 :- Agenda

- Why PMs must understand LLMs
- LLMs 101 — data → training → inference
- The AI economy & product landscape
- System components in AI products
- Hands-on with OpenAI Playground

How did we arrive here?



1950 – 2000

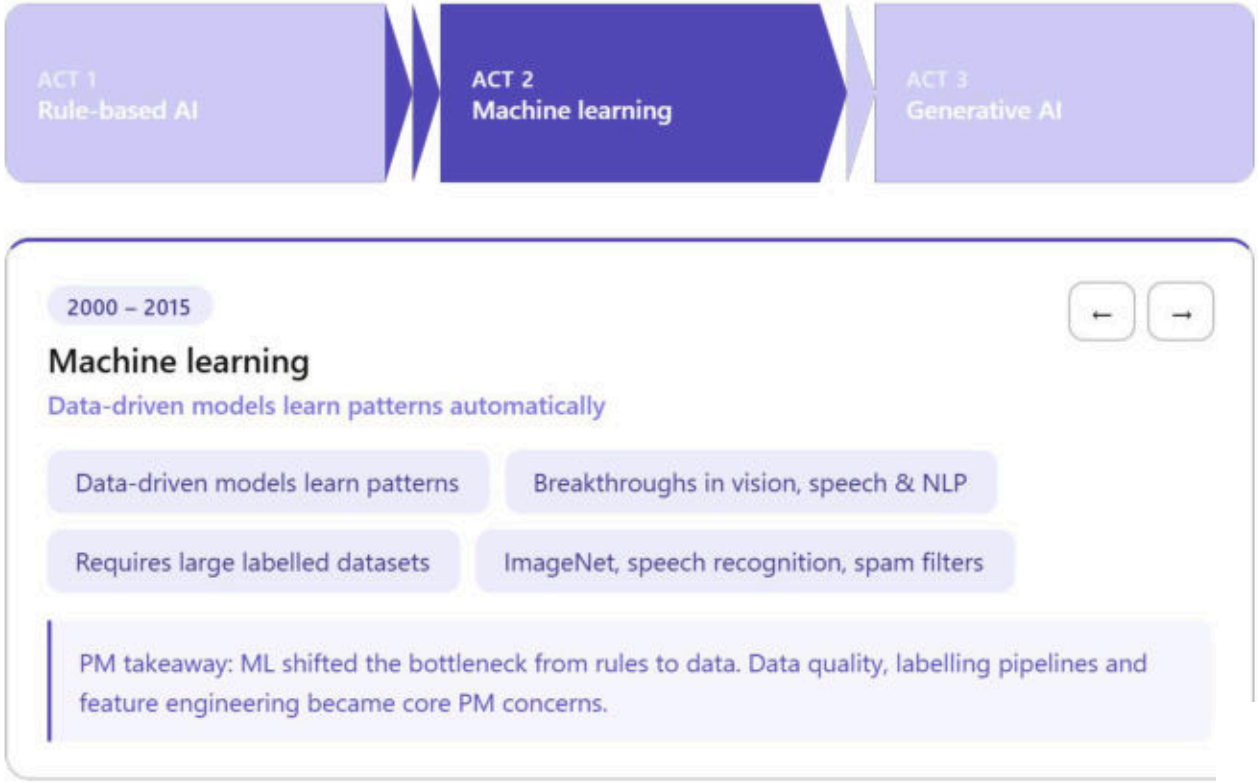
Rule-based AI

If this, then that — hand-crafted logic

- Expert systems with if-then rules
- Rigid, domain-specific logic
- Struggled with scale & adaptability
- IBM Deep Blue, early chatbots

> PM takeaway: These systems needed constant manual updates for every new scenario. Maintainability was the bottleneck — not compute.

How did we arrive here?



How did we arrive here?



2017 – Present

Generative AI

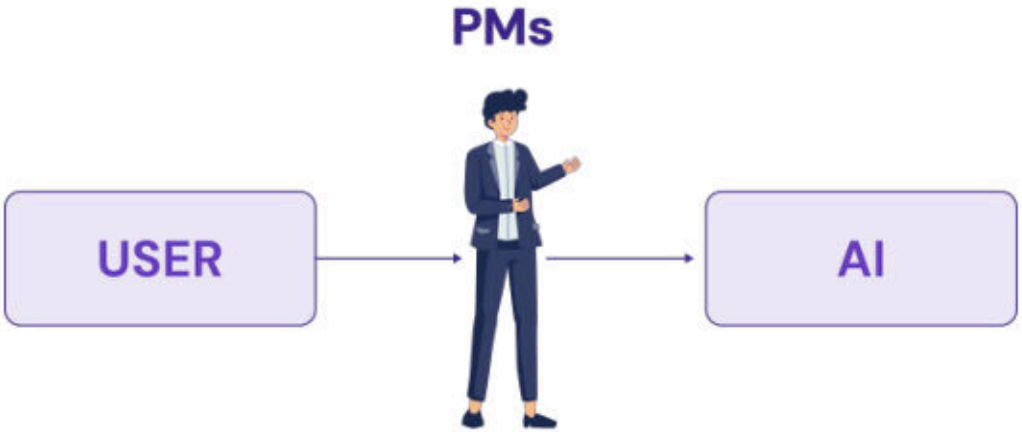
Creates text, images, code and acts autonomously

- Generates text, images, audio & code
- Prediction + reasoning across domains
- Acts autonomously through agents
- GPT-4, Claude, Gemini, Stable Diffusion

> PM takeaway: Gen AI is a product layer you design for — not just a feature. Prompting, RAG, safety and UX trust are now core PM skills.

Why PMs Must Understand LLMs

Key Idea: PMs bridge human needs and machine capabilities

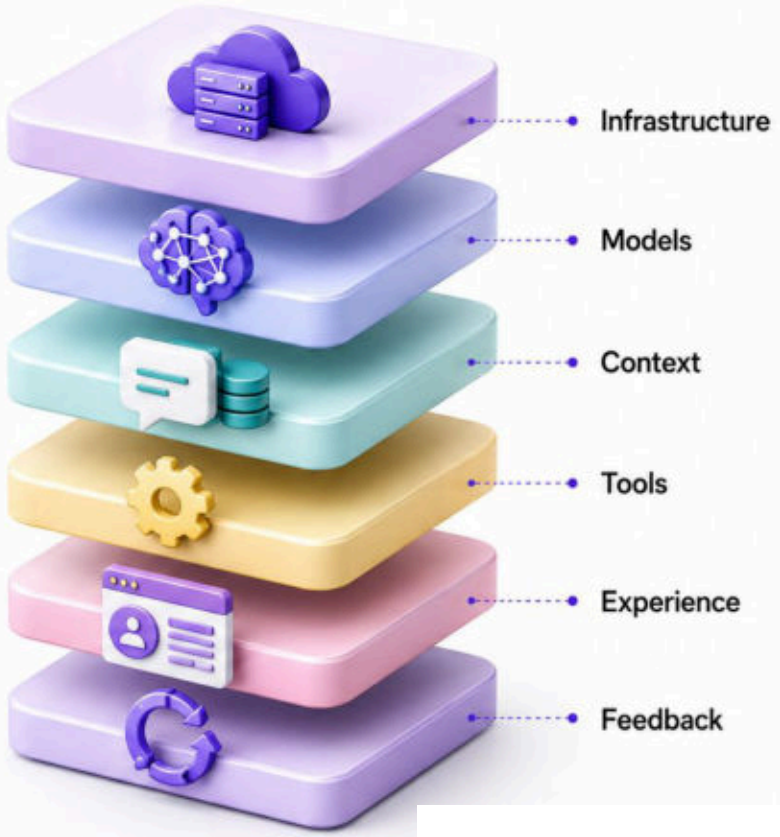


The AI Product Stack

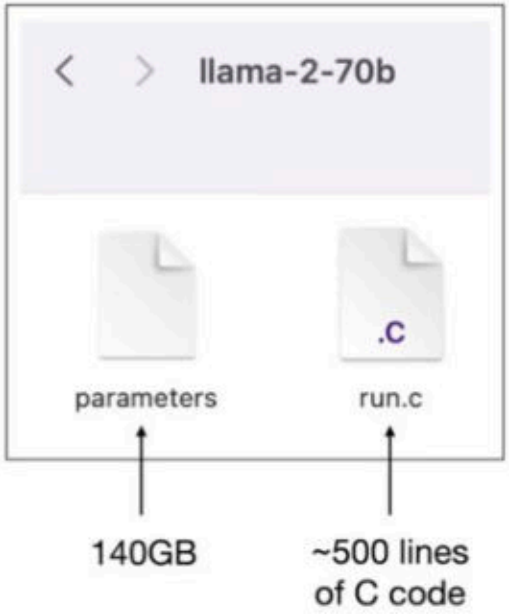
Where value is created in an AI product

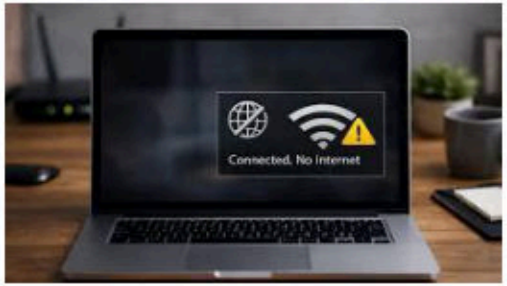
An AI product is not just a model.
It is a system made of:

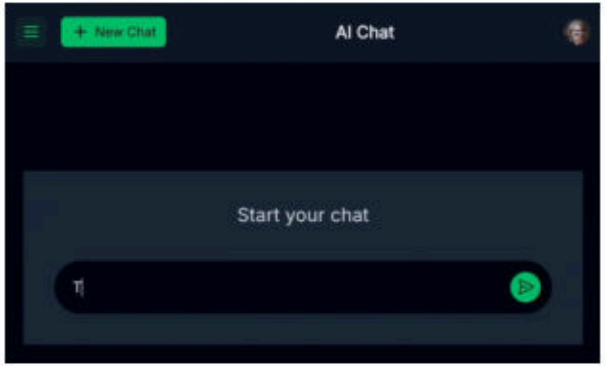
Infrastructure	Compute, storage, hosting
Models	Foundation models and specialised models
Context	Prompts, memory, retrieved knowledge
Tools	APIs, databases, actions, workflows
Experience	What the user sees and controls
Feedback	Signals that improve the product



LLMs 101 — What is it?







LLMs 101 — How They Work

- LLMs work as giant statistical prediction machines that repeatedly predict the next word in a sequence.
- They learn patterns in their text and generate language that follows those patterns.

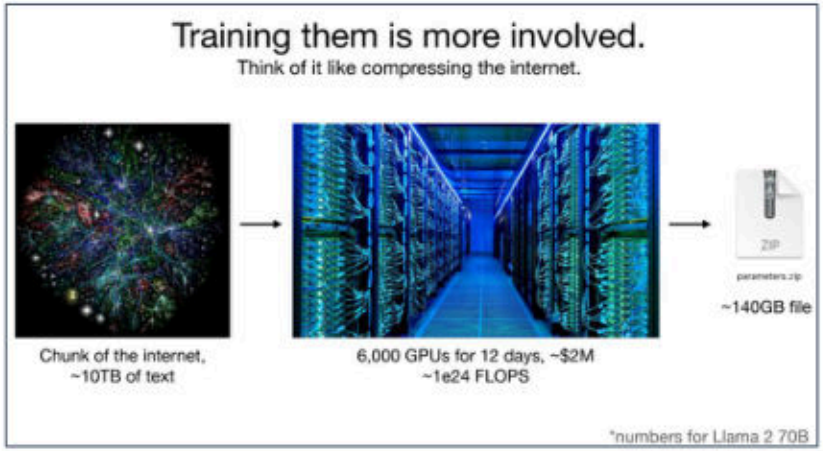
Where there's a will, there's a ….

A. **way** B. **smith** C. **document** D. **NOTA**

LLMs 101 — How They Work



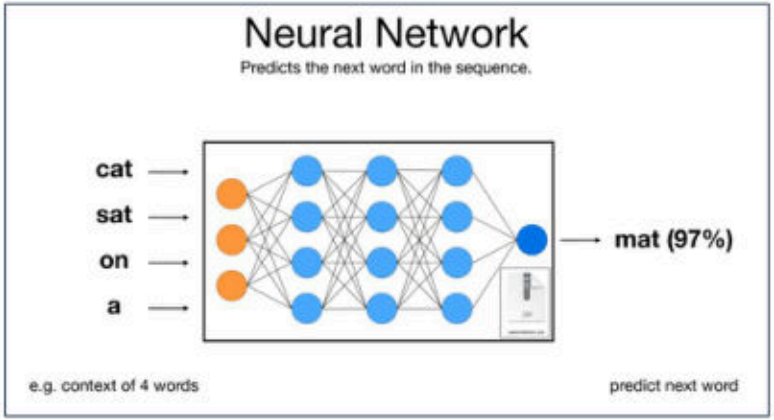
- **Trained on massive text datasets**
- Predict next word (probability, not truth)
- Controlled by parameters like temperature



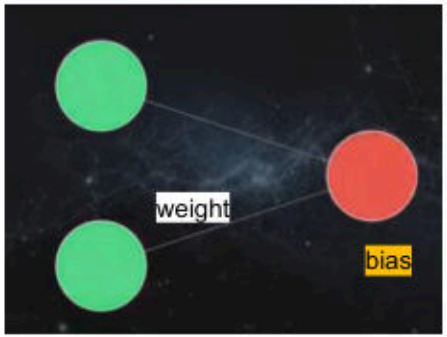
LLMs 101 — How They Work

Text → Tokenization → Neural Prediction → Output

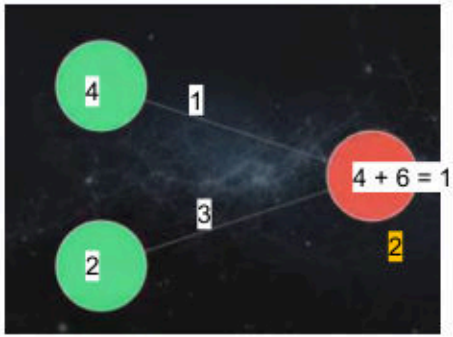
- Trained on massive text datasets
- **Predict next word (probability, not truth)**
- Controlled by parameters like temperature



Neural Network basics



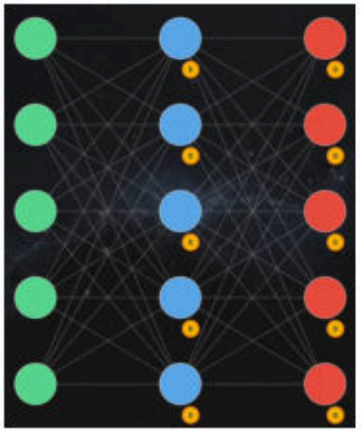
2 neurons
Input layer



1 neuron
Output layer

Final =12

Each neuron has bias
param added to result



5*5 + 5*5 + 10 = 60
parameters

Input value X weight = neuron activation

Weights: The model's learned importance values that determine how strongly each input influences the output.
Bias: An adjustable offset that helps the model shift the output up or down, even when inputs are zero.

Average LLM has 14 billion parameters!

Neural Network basics

Imagine you're predicting a **house price**.

Formula:

$$Price = 10000 \times Bedrooms + 50000$$

- **Weight = 10,000** → each extra bedroom increases the price by ₹10,000.
- **Bias = 50,000** → even a house with 0 bedrooms starts at ₹50,000 because the land itself has value.

For a 3-bedroom house:

Price = (10,000 × 3) + 50,000 = ₹80,000

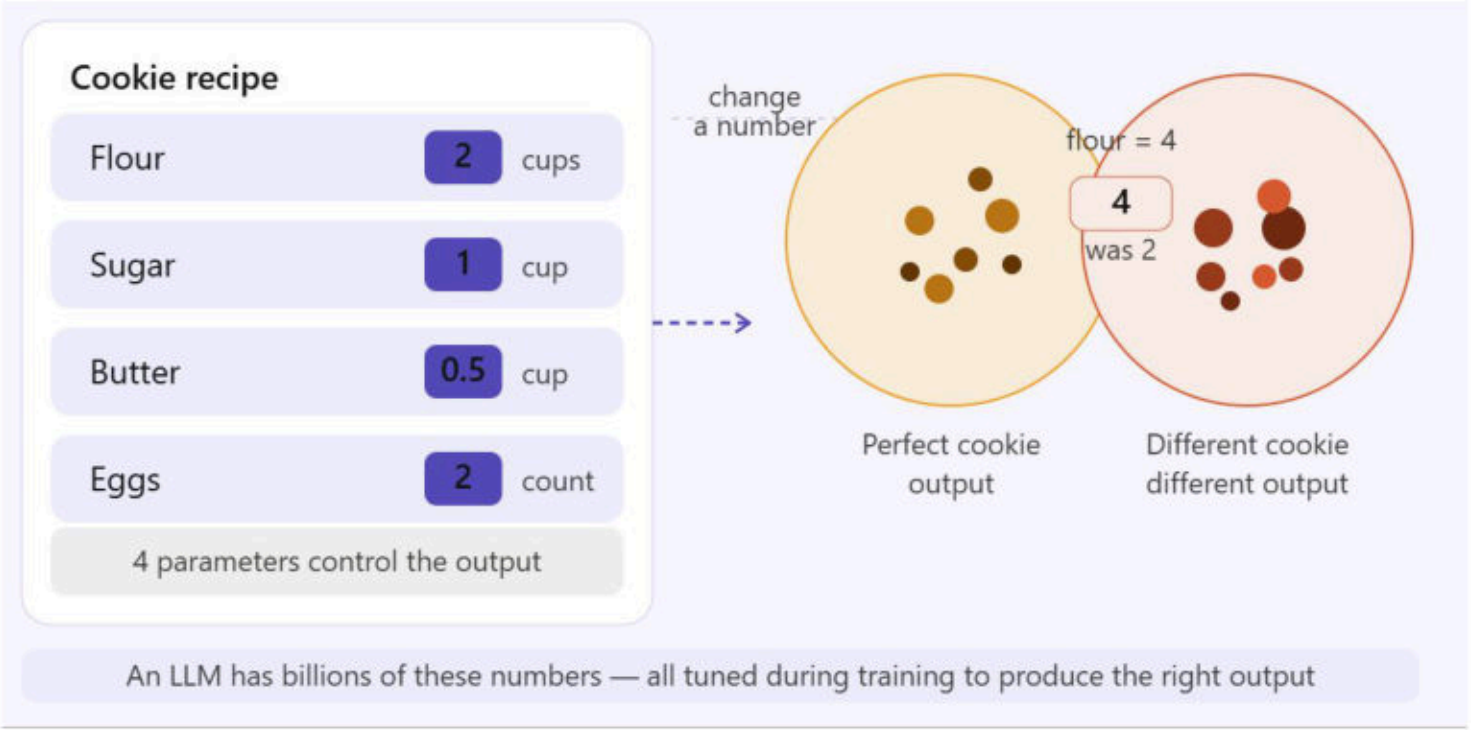
So:

- **Weight = "How much does this feature matter?"**
- **Bias = "Where do we start before looking at any features?"**

Typically LLMs are trained on:

- grammar
- reasoning patterns
- coding knowledge
- writing styles
- factual associations
- language structure

What are parameters?



When parameter #5,482 is set to 0.732, it helps the model recognise certain language patterns. Ch
— those patterns look different.

How does training set the parameters?

Guess → check → adjust. Trillions of times.

At the start of training, all parameters are random. The model gives garbage answers. Then this loop runs — over and over — for weeks on massive computers.

1. **Model sees a sentence**
   "The cat sat on the ___"
2. **Model guesses the next word**
   It says "banana." Wrong.
3. **Every parameter adjusts — just a tiny bit**
   Parameter 1 moves from 0.5430 → 0.5431. Tiny. But across billions of examples, it adds up.
4. **Repeat trillions of times**
   After enough repetitions the parameters are no longer random — they've learned how language works.

Nobody manually teaches your body to balance. You fall, you adjust, you try again. Training does the same thing — but with billions of numbers instead of your body.

More parameters = bigger model. Simple as that.

7B	70B	700B+
e.g. Mistral 7B	e.g. Llama 3 70B	GPT-4 scale
~14 GB · runs on a laptop · fast responses	~140 GB · needs a server · slower	~1.4 TB · data centre only · expensive

The name tells you the size. "Llama 3 70B" = 70 billion parameters. Bigger number → bigger file → more powerful computer needed → slower responses → more expensive to run.

AI & LLM Glossary for PMs

Token ⟶ A chunk of text (word / part of word) that LLMs process; e.g., “ChatGPT” → [Chat, G, PT]

Parameter ⟶ The internal “knob” or weight that decides how the model predicts; billions of them define model quality

Model ⟶ The trained brain that predicts the next token given context

AI & LLM Glossary for PMs

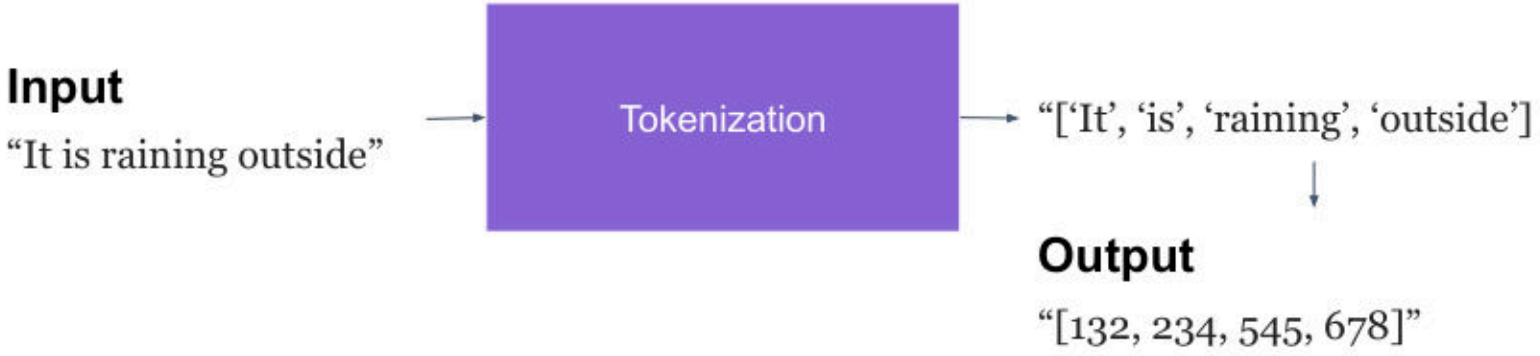
Temperature → Controls randomness — lower = precise, higher = creative

Context Window → How much text the model can "remember" in one go

NLP → Natural Language Processing — helps computers understand, interpret, and generate human language.

How LLMs understand text

Tokenization → Divide text into "tokens" and assign unique ids to each



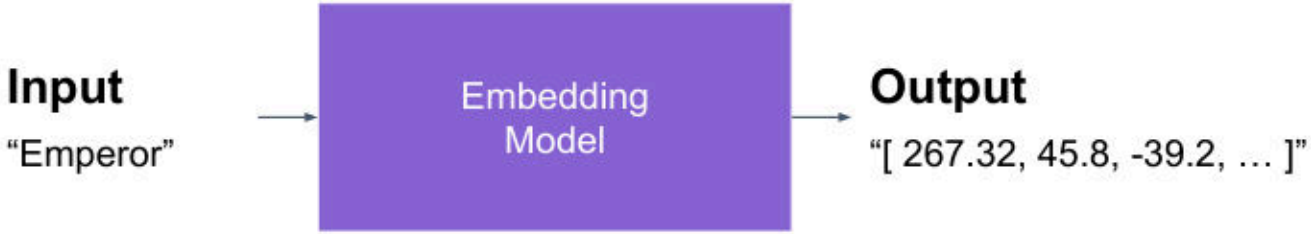
Try here: https://platform.openai.com/tokenizer

How LLMs understand text

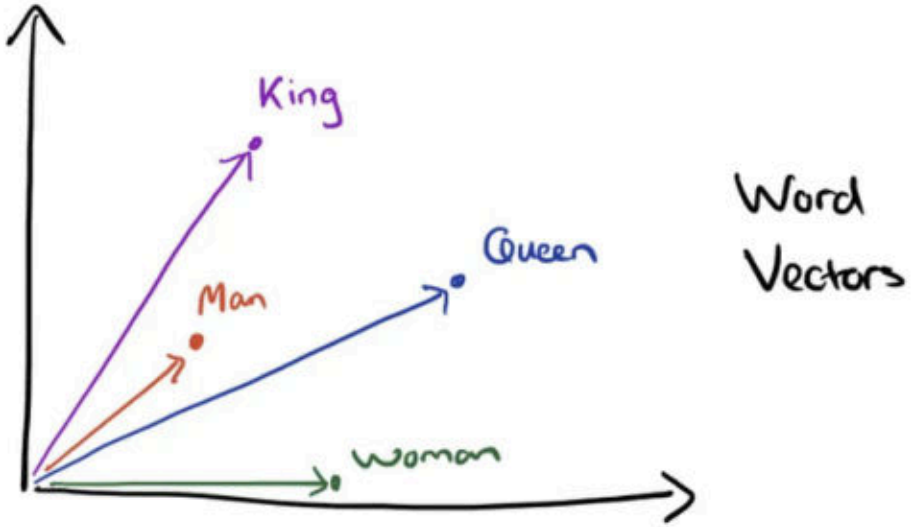
Embedding → Convert text/tokens into “vectors”

Vectors capture the semantic meaning of the text.

This allows AI to understand and work with language effectively.



How LLMs understand text



An example of word vectors and vector composition. Source: Adrian Colyer.

How LLMs understand text



An example of word vectors and vector composition. Source: Adrian Colyer.

How LLMs understand text



Example of different word vectors and their different dimensions. Source: Adrian Colyer.

See it for yourself on https://projector.tensorflow.org/

Common Pitfalls



Hallucination → False confidence



Bias → Data imbalance



Overfitting → Poor generalization

Non-Determinism Changes Product Design

The same input may not produce the same output.

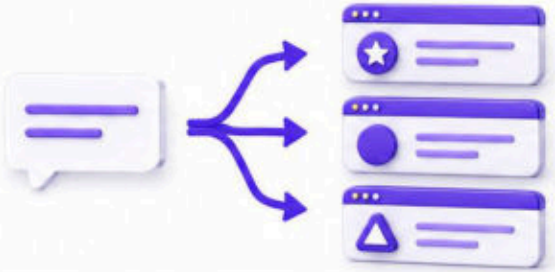
Traditional software

Same input → Same output



AI products

Same input → Different output



- Outputs may vary
- Edge cases are harder to predict
- Quality is measured across many examples
- Fallback behavior becomes part of the UX
- Users need visibility and control

PM question: How much variation can the user tolerate before trust breaks?

Designing for Uncertainty

AI products need confidence, recovery, and control

Show confidence carefully

Avoid presenting uncertain answers as facts.

Offer recovery paths

Let users retry, edit, verify, or escalate.

Make sources visible

Help users understand where the answer came from.

Keep humans in control

Require confirmation for sensitive or irreversible actions.



The Agency–Control Spectrum

How much should the AI decide?

1 Assist



Suggests, drafts, or summarises

2 Recommend



Ranks options and proposes actions

3 Act with approval



Executes after user confirmation

4 Act autonomously



Completes tasks without approval

Lower agency / lower risk

Higher agency / higher risk

PM decision: Choose the lowest level of autonomy that still creates meaningful value.

When to Increase AI Agency

Agency should grow with evidence

Increase autonomy when:

- The task is repetitive
- Errors are reversible
- Outcomes are measurable
- User intent is clear
- Historical performance is strong
- Guardrails are reliable



Keep human approval when:

- Financial or legal impact is high
- Actions affect other people
- Mistakes are difficult to reverse
- The system lacks enough context

Token Economics for PMs

Every AI interaction has a cost

- Input tokens
- Output tokens
- Model choice
- Context length
- Number of retries
- Retrieval volume
- Tool calls



Cost per task × Tasks per user × Active users = Product cost

Cost, Quality, and Latency

AI product decisions are three-way trade-offs



PM question:
What is the minimum quality threshold the user actually needs?

Model Selection Is a Product Decision

Choose the model for the task, not the brand

- Task accuracy
- Tool-use ability
- Response time
- Privacy requirements
- Cost per request
- Rate limits
- Context-window needs
- Regional availability
- Structured output reliability



PM question:
Which model best meets our product requirements at a sustainable cost?

The AI Economy

Who captures value in the AI ecosystem?

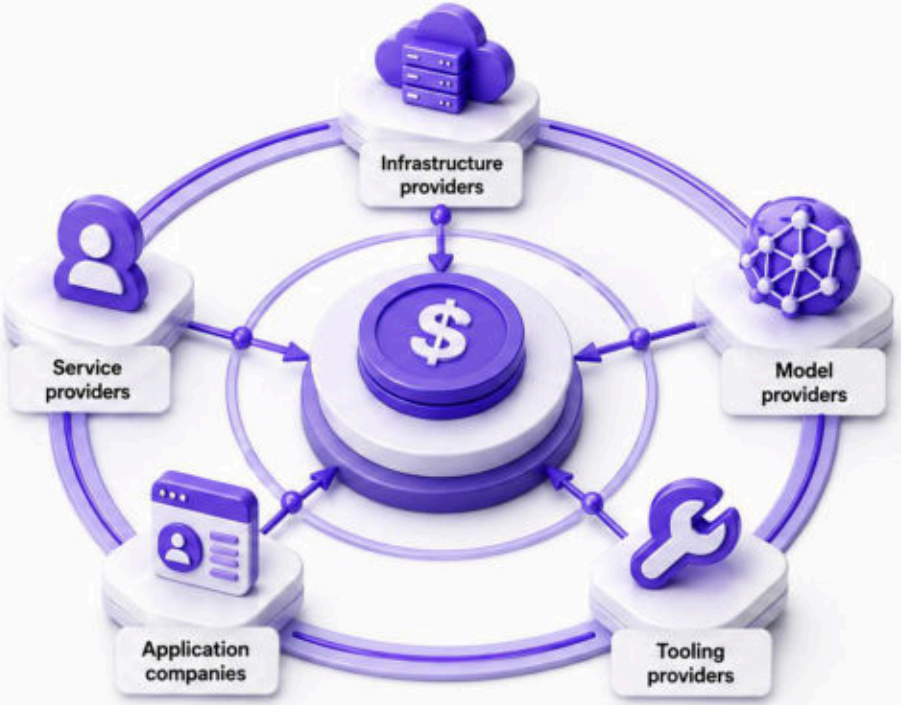
Infrastructure providers
Provide compute, chips, and cloud access.

Model providers
Build and serve foundation models.

Tooling providers
Enable orchestration, evaluation, data pipelines, and deployment.

Application companies
Solve a specific user problem using AI.

Service providers
Help companies implement, customise, and operate AI systems.



PM question: Where does your product create unique value that cannot be replaced by the model provider?

The AI Product System

Every AI product is a feedback system

The model is one component inside a larger product loop.

- User input defines the task
- Context shapes the response
- Feedback improves quality over time

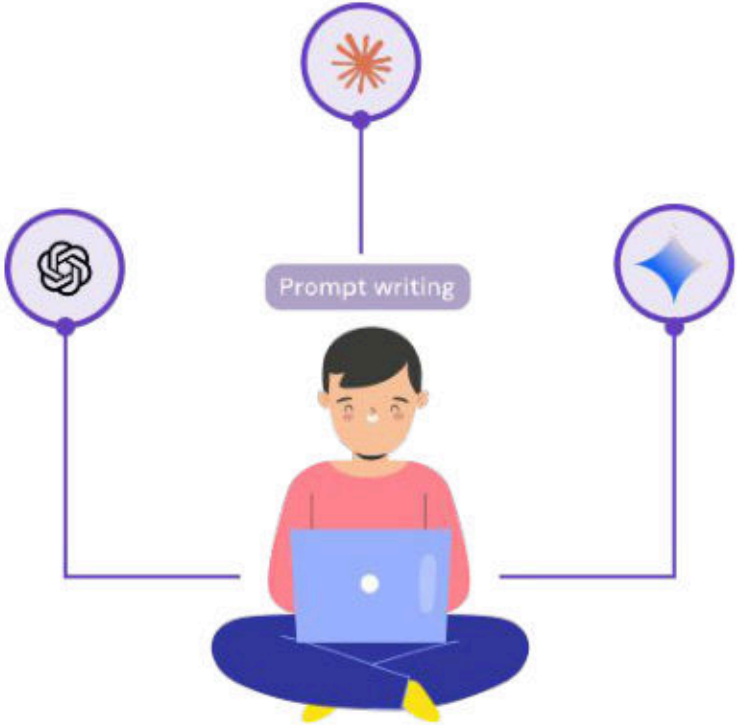


Session 1 Takeaways

- Explain LLM functioning in simple terms
- Identify AI product layers and components
- Use Playground to observe model behavior

Prompt Engineering & RAG for PMs

“From talking to AI → building with it.”



Day 2 :- Agenda

- Prompt Engineering Basics
- Iteration & testing for clarity
- Retrieval-Augmented Generation (RAG)
- Hands-on mini assistant build

Prompt Engineering for PMs

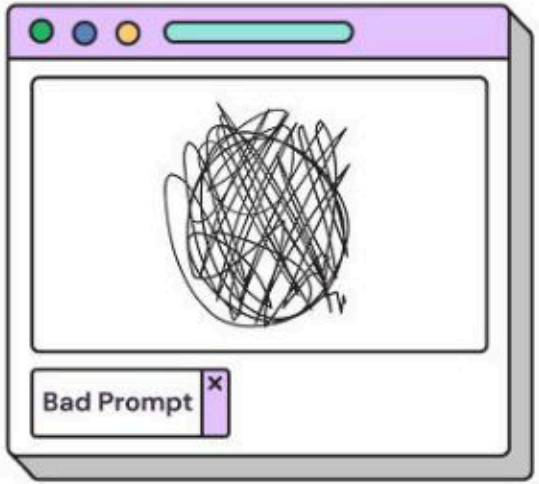
What is a Prompt? *"A prompt is your set of instructions to the AI."*

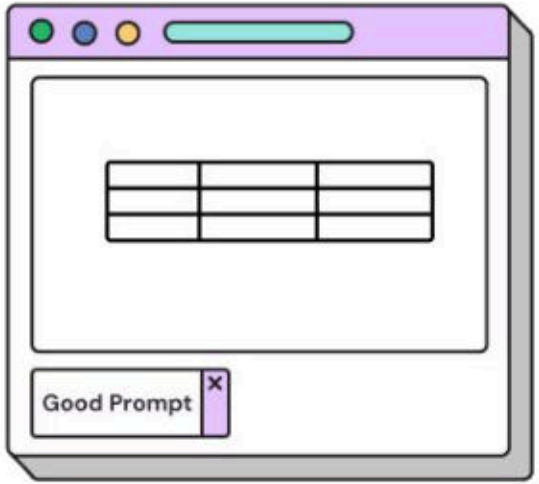
Think of a prompt as the creative brief you'd give to a designer or developer.

It's not just a question - it's a complete instruction set that guides the AI toward your desired outcome.

- **It's a brief — not a question**
  - Effective prompts provide direction and context, not just queries.
- **Contains role, context, and constraints**
  - Give the AI a perspective, background information, and boundaries to work within.
- **Quality depends on clarity and specificity**
  - The more precise your instructions, the more usable your output becomes.

Prompt Engineering for PMs





Prompt Quality Ladder

Each rung on this ladder adds clarity and strategic focus. The higher you climb, the more actionable and product-fit your AI responses become.



Vague

“Write about my product”



Clear

“Describe my SaaS tool for remote teams.”



Contextual

“Describe my SaaS tool for remote teams struggling with handoffs.”



Strategic

“Create a benefit-driven tagline for remote PM tools focused on async work.”

The PREP Framework

Purpose → Requirements → Examples → Parameters

"A practical, repeatable structure for every product prompt."

PREP helps you brief the AI with the same rigor you'd apply to writing a mini PRD. This framework ensures you never miss critical context, making your prompts consistent and your outputs reliable

Input: Your PREP Prompt

Purpose: Describe a new feature for our team app.

Requirements: Focus on speed and teamwork.

Example: "Assign, track, and celebrate milestones - all in one view."

Parameters:
- Tone = upbeat
- Length = 3 sentences max

Output: AI Result

Bring your team together with instant task assignments and real-time progress tracking. Celebrate wins as they happen with automatic milestone notifications that keep everyone motivated. Get the full picture in one collaborative view - no more jumping between tools.

The PREP framework

Purpose

What do you want the AI to create? Be specific about the deliverable.

Requirements

What must the output include? Define tone, audience, key themes, or constraints.

Examples

Show the AI what "good" looks like with 1-2 sample phrases or formats.

Parameters

Set boundaries: length, format, style, or any other specifications.

CRISPE Framework

- **C - Context (Set the scene. Tell the AI:)**
  - What you do
  - What the situation is
  - Your goals
  - Who the audience is
- **R - Role (Assign the AI a clear identity)**
  - "Act as a senior marketing strategist for SaaS startups"
  - "Be a UX designer who specializes in mobile onboarding flows"
- **I - Instruction (Explain the task and why it matters)**
  - What you want done
  - Why you're doing it
  - What success looks like
- **S - Specification (Explain how you want the result delivered)**
  - Format (bullets, slides, paragraphs)
  - Length
  - Tone and voice
  - Must-have elements
- **P - Performance (Set clear quality standards)**
  - Success criteria
  - Metrics to hit
  - What to avoid
  - Industry standards
- **E - Example (Show what good looks like)**
  - Sample content
  - Previous success
  - Competitor examples
  - Specific elements to replica

CRISPE Framework Example

C - Context
I'm a career coach + PM mentor creating simple, high-engagement LinkedIn posts for aspiring PMs.

R - Role
Act as a senior LinkedIn content strategist who writes scroll-stopping short posts.

I - Instruction
Write one LinkedIn post based on <my idea>. Make it clear, relatable, and built for high retention.

S - Specification

- Hook under 8 words
- Second line starts with "I" + a number
- Short sentences, grade-4 English
- No emojis, no jargon
- Clean spacing + fast reading
- Power close under 8 words + a broad question

P - Performance
Success = simple, scannable, engaging.
Avoid long paras, fluff, corporate tone.

E - Example
Success hides in boring habits
I wasted 6 months learning this
…
Small wins move mountains.
What's one habit that changed you?

A word on “Tone”

Tone is one of the most powerful parameters in prompt engineering. The same product can speak to completely different audiences simply by adjusting how you describe it.

Tone switching is the fastest way to align copy with your brand voice and audience expectations. Test multiple tones to find what resonates.

Example:
Describe our AI email assistant in three tones:
1. Professional
2. Playful
3. Inspirational

Professional

"An AI assistant that drafts efficient, brand-correct emails in seconds."

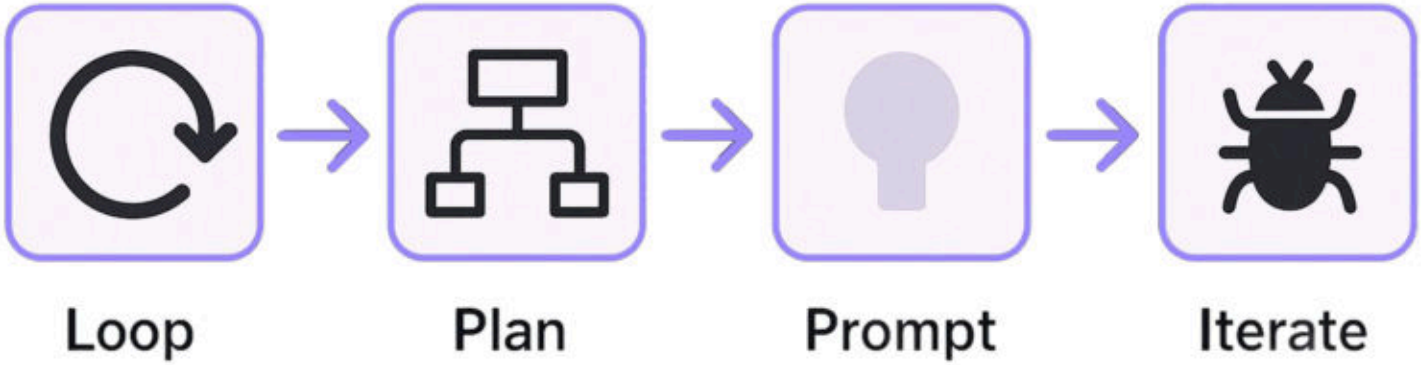
Playful

"Your cheeky inbox sidekick — writes emails so you don't have to!"

Inspirational

"Unlock clarity in your communications and connect with purpose."

Iteration & Testing



Prompting vs RAG vs Fine-Tuning

Choose the right way to improve the output

Prompting

Use when the model needs clearer instructions.

RAG

Use when the model needs access to updated or private knowledge.

Fine-Tuning

Use when the model needs consistent behaviour, tone, or task patterns.



These approaches solve different problems. Use the right lever for the right issue.

The PM Decision Tree

How should we improve this AI feature?



Use this tree to choose the right lever for improving an AI feature with the highest impact and lowest risk.

1. Is the output unclear or inconsistent? → Improve the prompt
2. Is the answer missing current or company-specific knowledge? → Add RAG
3. Is the required behaviour repeated, specialised, and stable? → Consider fine-tuning
4. Is the task still unreliable? → Revisit workflow, data, or model

The EVAL Framework

A practical way to define and improve AI quality



E | Expected outcome

What should the AI do for the user?

Define the intended task, user value, and acceptable behaviour.

V | Validation criteria

How will we know the output is good?

Set measurable checks for accuracy, relevance, faithfulness, clarity, latency, and safety.

A | Adversarial testing

Where could the system fail?

Test am
informa
edge ca

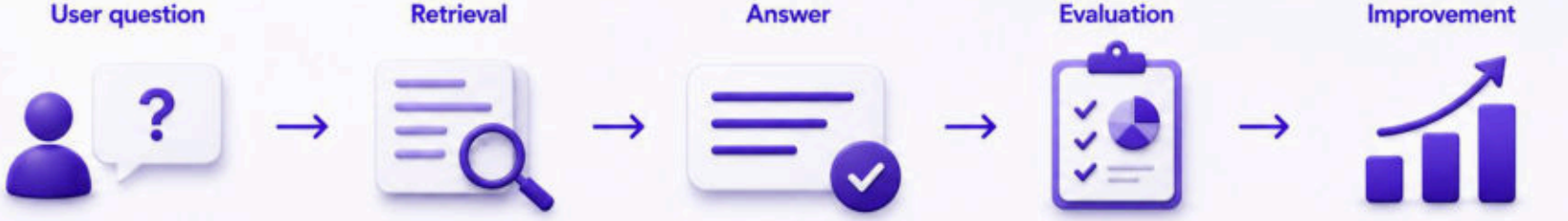
L | Learning loop

What changes after a failure?

Improve the prompt, retrieval, model, guardrails, or product workflow based on observed failures.

EVAL in Practice

Evaluating a product-support assistant



Where it can fail (A) and how we fix it (L)

Retrieval issues

- Outdated or missing docs
- Conflicting information
- Poor search or chunking

Fix by: Improving chunking, search, indexing, or metadata.

Generation issues

- Hallucinations or unsupported claims
- Irrelevant or incomplete answers
- Weak reasoning or prompt issues

Fix by: Improving prompt, context selection, model choice, or guardrails.

UX / behavior issues

- Unclear answers or jargon
- Missing citations or context
- No refusal or clarification flow

Fix by: Improving clarification flows, citations, refusal states, and UX

Evaluating Retrieval

Did the system retrieve the right information?

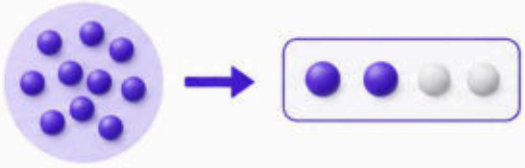
Precision

Of the retrieved items,
how many were relevant?



Recall

Of all relevant items,
how many did the system retrieve?





- **High precision, low recall:** accurate but incomplete
- **Low precision, high recall:** broad but noisy
- **High precision, high recall:** relevant and comprehensive

Evaluating the Final Answer

Did the model use the evidence correctly?

Faithfulness
Is the answer supported by the retrieved context?

Relevance
Does the answer directly address the user's question?

Completeness
Does it include the important information?

Clarity
Is it understandable and actionable?

Safety
Does it avoid harmful or unsupported claims?



LLM-as-a-Judge

Using an LLM to evaluate another LLM

An evaluator model can score outputs against a rubric.

It can assess:

- relevance
- faithfulness
- completeness
- tone
- formatting
- policy compliance



But it also has limitations:

- evaluator bias
- inconsistent scoring
- preference for longer answers
- disagreement with human judgement

Build an Evaluation Dataset

Do not evaluate with random prompts

Include

- Common user questions
- Difficult edge cases
- Ambiguous requests
- Unsupported questions
- Adversarial inputs
- Examples with known correct answers

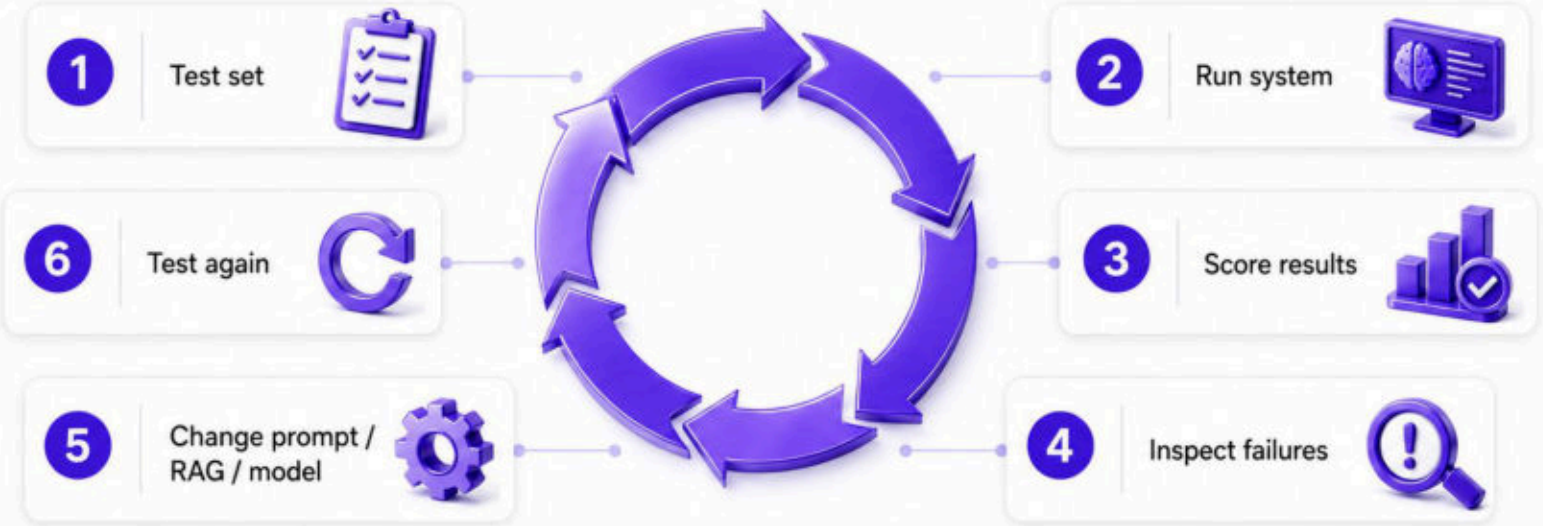
For each test case define

- Expected behaviour
- Acceptable answer
- Failure condition
- Severity of failure



The AI Evaluation Loop

Build, test, diagnose, improve



Evaluation should happen before launch, after major changes, and when data or models change.

Hands-on: Compare Model Behaviour

Observe how model settings change the output

Use OpenAI Playground or Claude Workbench.

Test the same prompt with changes to:

- model
- temperature
- system instruction
- output format
- context length
- examples

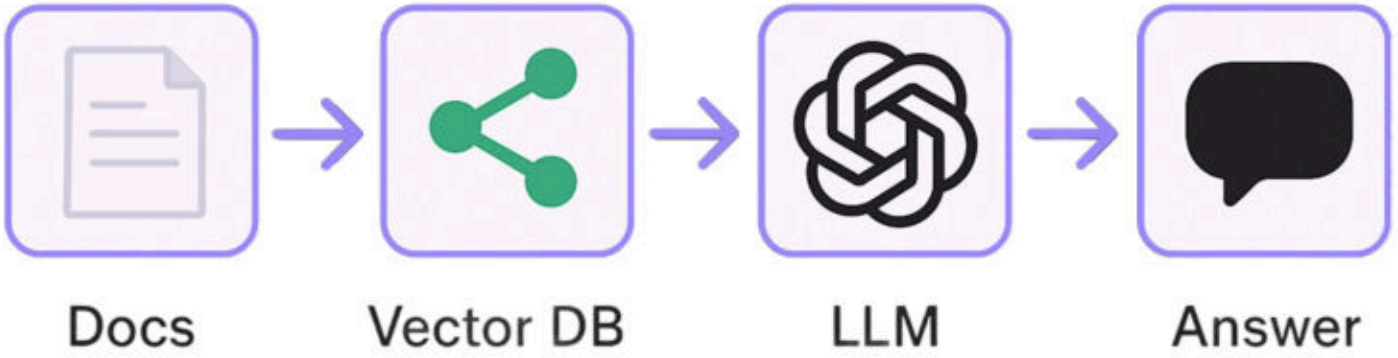


Record: quality, consistency, latency, cost, failure patterns

Output: A short recommendation on which setup is best for the use case.

What Is RAG

Definition: Retrieval + Generation = Grounded answers.



Why RAG Matters for PMs

Scenario	Benefit
Support chatbot	Consistent answers from docs
Product research	Faster competitor insights
Internal tools	Instant knowledge retrieval

RAG Pipeline Simplified



What Good RAG Actually Requires

RAG quality depends on more than retrieval

- Clean source documents
- Good chunking
- Meaningful embeddings
- Accurate retrieval
- Relevant context selection
- Grounded generation
- Source citations
- Evaluation and monitoring



RAG does not automatically remove hallucinations.

RAG Failure Modes

Where RAG systems break

1 Retrieval failure



The right information was not found.

2 Ranking failure



Relevant information was retrieved but placed too low.

3 Context failure



Too much, too little, or conflicting information was passed.

4 Generation failure



The model ignored or misread the retrieved content.

5 Source failure



The original document was incomplete or outdated.

Which layer caused the wrong answer?

Leveraging AI for Product Discovery



Build a Mini AI Product Assistant

Task: Conceptually design a RAG flow that answers product queries.

- Input: FAQ data or docs
- Retrieval: contextual search
- Output: clear answer in Slack or Sheet

Assignment Brief

Objective

Practice designing a reusable system prompt and then using it to produce a real deliverable, demonstrating that good prompt engineering is really a form of process design.

Step 1: Design the System Prompt

Step 2: Generate the PRD

Step 3: Prepare Your Submission

Submission guidelines

- Your submission must be in .PDF file format only.
- Your submission must also include a video in .MP4 file format only.
- Make sure you are visible in the video along with your presentation and the video must strictly not exceed 5 minutes
- You may use AI tools to brainstorm or organize your thinking
- Do NOT only use AI to generate your final output. Your submission must reflect your reasoning, structure, and originality
- Please submit the Loom video link, or alternatively, share the video via a Google Dri

Step 1: Design the System Prompt
Design a system prompt that instructs an LLM to act as an expert Product Manager / PRD writer.

Your system prompt should specify:

- The persona/role the model should adopt (e.g., *"Senior PM with 10+ years in B2B SaaS"*).
- The exact PRD structure/sections it must produce (e.g., Problem Statement, Goals & Non-Goals, User Personas, User Stories, Success Metrics, Scope, Timeline, Risks/Dependencies, Open Questions).
- Tone and depth expectations (concise vs. exhaustive, technical vs. business-facing).
- Formatting rules (headers, tables for metrics, numbered user stories, etc.).
- Guardrails—what it should ask the user for before writing, what it should flag if information is missing, and what it should not do (e.g., don't invent metrics, don't skip edge cases).
- Clarifying-question behavior (should it ask 3–5 scoping questions before drafting, or draft with stated assumptions?).

Step 2: Generate the PRD
Use your system prompt to generate a complete Product Requirements Document (PRD) for a product of your choice.

The product can be:

- A real product you are working on, or
- A hypothetical feature.

Step 3: Prepare Your Submission
Submit both artifacts:

- The system prompt itself.
- The resulting PRD it generated.

Day 2 Takeaways

- Write effective prompts for clarity & control
- Understand how RAG grounds AI answers
- Use AI for research and discovery

Assignment

Brief

"PulseReel," a short-video app with 60M Indian users aged 18–25, just watched its recommendation algorithm turn a single unverified "incident" reel into a nationwide protest movement within 72 hours. Government scrutiny of algorithmic amplification is rising, and PulseReel's leadership wants a way to detect and handle fast-mobilizing content differently from ordinary viral content, before regulation forces a clumsier fix on them. Define a version 1 solution that does this without turning PulseReel into a de facto censor or killing the organic reach that makes it valuable. What do you build, and where's the line?

Use the PRD system instruction to work through and structure your solution.

Submission guidelines

- Your submission must be in .PDF file format only.
- Your submission must also include a video in .MP4 file format only.
- Make sure you are visible in the video along with your presentation and the video must strictly not exceed 5 minutes
- You may use AI tools to brainstorm or organize your thinking
- Do NOT only use AI to generate your final output. Your submission must reflect your reasoning, structure, and originality
- Please submit the Loom video link, or alternatively, share the video via a Google Drive link.



Thank you!